

Resumen Tecnico de la Integracion con JFlex y CUP

Proyecto: Procesamiento de Lenguaje Natural Basico | Unidad 2

1. Objetivo del Cambio

Se actualizo el backend del proyecto para reemplazar el analizador lexico y el parser escritos manualmente por analizadores generados formalmente con **JFlex** y **CUP**. Con este cambio, el programa mantiene el mismo funcionamiento visual y la misma API REST, pero el proceso interno de analisis queda basado en herramientas propias de compiladores y teoria de lenguajes.

Flujo actual:

```
Oracion del usuario
-> NormalizadorOracion
-> LexerCup generado por JFlex
-> ParserCup generado por CUP
-> ResultadoAnalisis
-> DetectorAmbiguedad
-> API REST
-> Frontend React
```

2. Cambios Realizados

1. Se configuro Maven para generar automaticamente las clases necesarias de JFlex y CUP durante la fase `generate-sources`.
2. Se creo el archivo `LexerCup.flex`, donde se define el analizador lexico con JFlex.
3. Se creo el archivo `ParserCup.cup`, donde se define la gramatica libre de contexto con CUP.
4. Se agrego `AnalizadorJFlexCup.java` para conectar el lexer generado con el parser generado.
5. Se agrego `ConstructorResultadoCup.java` para construir el resultado final que consume el frontend.
6. Se actualizo `AnalizarOracionUseCase.java` para usar el nuevo analizador con JFlex y CUP.
7. Se mantuvo el detector de ambiguedad, porque esa funcionalidad trabaja despues de validar la oracion.
8. Se eliminaron las clases manuales `Lexer.java`, `Parser.java` y `Fragmento.java`, ya que fueron reemplazadas por la nueva integracion.
9. Se actualizo la documentacion del proyecto en el `README.md`.
10. Se ajusto el comando del frontend para que Vite muestre un solo enlace local.

3. Ubicacion de los Archivos Principales

Archivo	Funcion
<code>backend/pom.xml</code>	Configura Spring Boot, Java, JFlex, CUP y los plugins de generacion.

Archivo	Funcion
backend/src/main/jflex/LexerCup.flex	Define el analizador lexico. Reconoce palabras y las convierte en tokens.
backend/src/main/cup/ParserCup.cup	Define el parser sintactico y la gramatica libre de contexto.
backend/src/main/java/com/tfa/unidad2/dominio/servicios/AnalizadorJFlexCup.java	Orquesta el proceso completo: limpiar texto, tokenizar con JFlex, parsear con CUP y devolver el resultado.
ConstructorResultadoCup.java	Construye tokens, sujeto, verbo, complemento, derivacion, arbol y errores.
NormalizadorOracion.java	Limpia espacios y signos externos antes del analisis.
FragmentoSintactico.java	Transporta subarboles, tokens y patrones reducidos por CUP.
SintagmaVerbalCup.java	Agrupar el verbo y el complemento reducidos por el parser.
AnalizarOracionUseCase.java	Ejecuta el caso de uso principal y conserva la deteccion de ambigüedad.
DetectorAmbigüedad.java	Detecta casos ambiguos ya conocidos y genera explicaciones con arboles alternativos.
backend/src/test/java/.../AnalizadorOracionTest.java	Valida oraciones correctas, incorrectas, errores lexicos, derivacion, arbol y ambigüedad.
README.md	Documenta el flujo, la gramatica, los tokens, comandos y responsabilidades.

4. Funcionamiento del Analizador Lexico

El archivo `LexerCup.flex` define el analizador lexico. Este recibe la oracion limpia y separa cada fragmento de texto. Luego consulta la gramatica del proyecto para clasificar cada lexema.

Los tokens reconocidos son:

```
ART    -> articulos
PRON   -> pronombres
SUST   -> sustantivos
V      -> verbos
PREP   -> preposiciones
ADV    -> adverbios
ERROR  -> lexemas no permitidos o no reconocidos
```

Ejemplo:

```
El perro corre rapido

El      -> ART
perro   -> SUST
corre   -> V
rapido  -> ADV
```

5. Funcionamiento del Parser Sintactico

El archivo `ParserCup.cup` contiene la gramatica libre de contexto que valida la estructura de las oraciones. CUP toma los tokens producidos por JFlex y verifica si pueden formar una oracion valida.

```
S  -> SN SV
SN -> ART SUST
SN -> PRON
SN -> SUST
SV -> V
SV -> V C
C  -> ADV
C  -> SN
C  -> PREP SN
C  -> ADV C
C  -> SN C
C  -> PREP SN C
```

6. Resultado que se Devuelve al Frontend

La interfaz React no se cambio en su estructura principal, porque el backend sigue devolviendo el mismo tipo de respuesta JSON. El resultado incluye:

- Si la oracion es valida o invalida.
- Mensaje de estado o error.
- Sujeto identificado.
- Verbo identificado.

- Complemento, si existe.
- Tabla de tokens y lexemas.
- Derivacion por la izquierda.
- Arbol sintactico.
- Informacion de ambigüedad cuando aplica.

7. Manejo de Errores

El sistema conserva mensajes claros para el usuario. Los errores lexicos se detectan cuando aparece un numero, un simbolo no permitido o una palabra que no pertenece al vocabulario. Los errores sintacticos se detectan cuando la secuencia de tokens no cumple la gramatica definida en CUP.

Ejemplos de codigos de error:

```
ORACION_VACIA
NUMERO_NO_PERMITIDO
SIGNO_NO_PERMITIDO
PALABRA_NO_RECONOCIDA
FALTA_SUJETO
FALTA_VERBO
ORDEN_INCORRECTO
CONCORDANCIA_INVALIDA
COMPLEMENTO_INVALIDO
```

8. Pruebas Realizadas

Se ejecutaron las pruebas automatizadas del backend para comprobar que la integracion con JFlex y CUP mantiene el comportamiento esperado.

```
cd backend
mvn clean test
```

Resultado esperado:

```
Tests run: 29, Failures: 0, Errors: 0, Skipped: 0
```

Tambien se verifico el build del frontend:

```
cd frontend
npm run build
```

9. Comandos para Ejecutar el Proyecto

Backend

```
cd backend
mvn spring-boot:run
```

URL del backend:

```
http://localhost:5000
```

Frontend

```
cd frontend  
npm run dev
```

URL del frontend:

```
http://localhost:5173
```

10. Explicacion Corta para Sustentar

Se reemplazo el analizador manual por una integracion formal con JFlex y CUP. JFlex se encarga del analisis lexico, convirtiendo cada palabra en un token. CUP se encarga del analisis sintactico, validando esos tokens con la gramatica libre de contexto del proyecto. Luego se construye el mismo resultado que ya usaba el frontend: tabla de tokens, derivacion, arbol, sujeto, verbo, complemento y errores. De esta manera, el programa conserva la misma interfaz, pero internamente usa analizadores generados.

11. Estado Final

Los cambios fueron probados, documentados y subidos al repositorio de GitHub. El proyecto queda listo para ejecutarse con Maven y Vite, y los archivos generados por JFlex y CUP se crean automaticamente al compilar.